RELATÓRIO TÉCNICO

Revisão e adaptação dos módulos Extract / Transform / Load

Fase 5 — Adaptação da ETL incremental

Item	Descrição
Projeto	Data Warehouse AdventureWorks com ETL Incremental e Apache Airflow
Origem	SQL Server — AdventureWorks2016
Destino	PostgreSQL — adventureworks_dw
Schema de destino	dw
Atividade	Revisar e adaptar módulos Extract/Transform/Load
Data do registro	27/10/2026

Status: CONCLUÍDA

1. Objetivo

A tarefa teve como objetivo revisar formalmente os módulos de extração, transformação e carga da ETL Python, confirmando a compatibilidade com o modelo dimensional validado na Fase 4 e identificando eventuais resquícios do projeto anterior ou incompatibilidades de tipos e regras de carga.

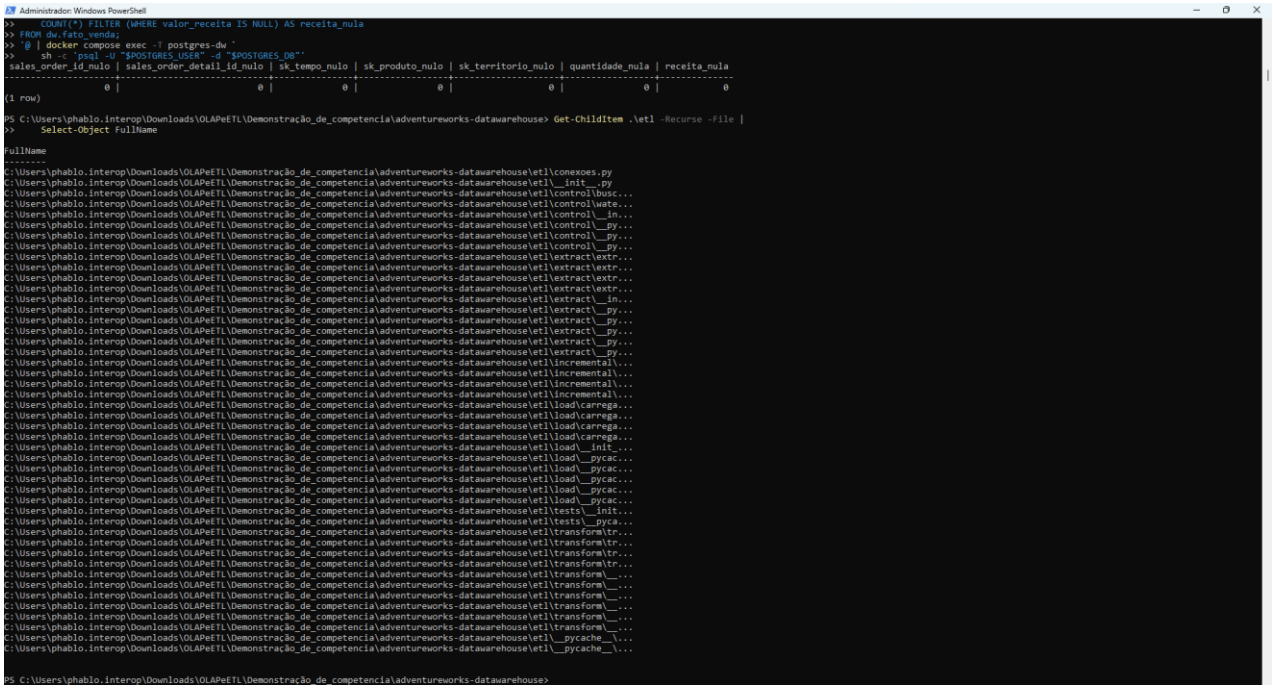
A revisão contemplou estrutura de arquivos, compilação sintática, origem e destino dos dados, regras incrementais presentes nos módulos de extração, padronização e tratamento de dados nas transformações e estratégias de idempotência/atualização nos loaders.

2. Estrutura e validação sintática

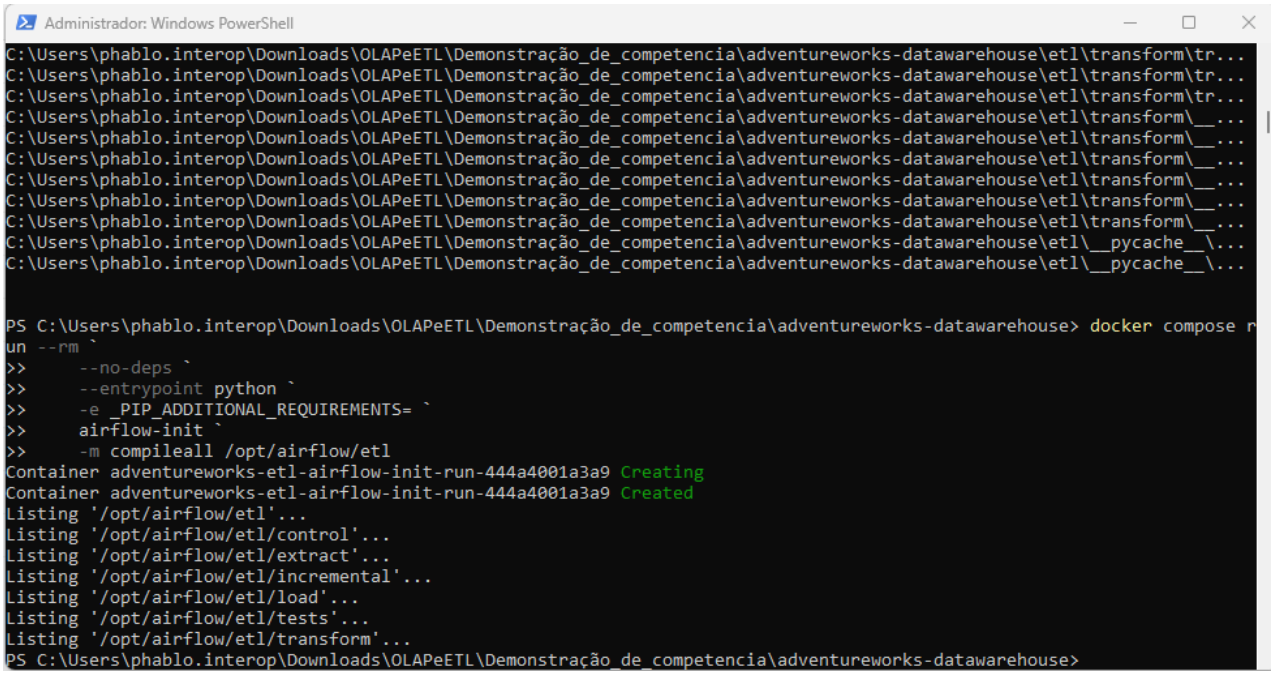
O pacote ETL encontra-se organizado em módulos especializados: control, extract, incremental, load, tests e transform, além do módulo conexoes.py. A estrutura foi inventariada e, em seguida, todo o pacote foi compilado com python -m compileall.

Bloco	Finalidade
extract	Leitura dos dados do AdventureWorks no SQL Server
transform	Padronização, limpeza, SKs e cálculos do modelo dimensional
load	Carga no PostgreSQL com regras de idempotência/SCD
control	Watermark e resolução das chaves dimensionais
incremental	Orquestração da lógica incremental
tests	Estrutura reservada para testes

O compileall percorreu todos os diretórios sem SyntaxError, IndentationError ou traceback, confirmando a validade sintática do pacote no ambiente Docker do projeto.



Inventário da estrutura do pacote ETL.



Validação sintática de todo o pacote ETL com compileall.

3. Revisão dos módulos Extract

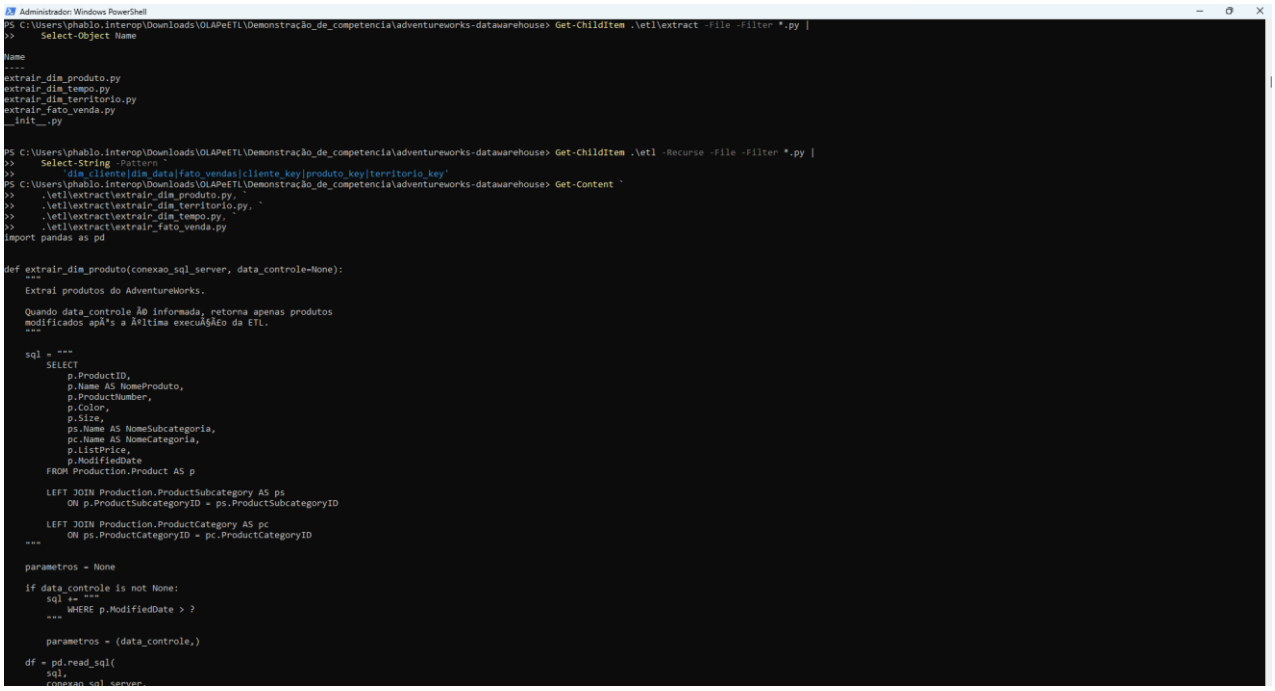
Foram confirmados quatro módulos de extração, alinhados às três dimensões e à tabela fato do modelo final.

Módulo	Origem principal	Aspectos validados	Status
extrair_dim_produto.py	Production.Product	Subcategoria, categoria e ModifiedDate	Aprovado
extrair_dim_territorio.py	Sales.SalesTerritory	Território e ModifiedDate	Aprovado
extrair_dim_tempo.py	Sales.SalesOrderHeader	MIN/MAX OrderDate e calendário diário	Aprovado
extrair_fato_venda.py	SalesOrderHeader + SalesOrderDetail	Grão de item, custo histórico/fallback e ModifiedDate	Aprovado

A busca textual por nomes do modelo antigo — dim_cliente, dim_data, fato_vendas, cliente_key, produto_key e territorio_key — não retornou ocorrências nos arquivos Python, indicando que a migração de nomenclatura foi consolidada.

Na fato, o custo unitário prioriza Production.ProductCostHistory e utiliza Production.Product.StandardCost como fallback. O ModifiedDate incremental é calculado a partir do maior valor entre SalesOrderHeader.ModifiedDate e SalesOrderDetail.ModifiedDate.

Observação técnica: uma alteração isolada em ProductCostHistory não provoca, por si só, a reextração de uma venda já carregada. Para a base AdventureWorks estática usada no trabalho, a estratégia adotada é suficiente e a limitação deve apenas ser documentada.



Auditoria dos módulos Extract e busca por referências do modelo antigo.

4. Revisão dos módulos Transform

Os módulos de transformação foram revisados quanto à nomenclatura final das colunas, coerção de tipos, tratamento de nulos, remoção de duplicidades, tradução da dimensão tempo, resolução de chaves substitutas e cálculos financeiros.

Módulo	Principais regras	Status
transformar_dim_produto.py	Renomeia campos, trata nulos, preço e deduplica por product_id_oltp	Aprovado
transformar_dim_territorio.py	Renomeia campos, trata nulos e deduplica por territory_id_oltp	Corrigido/Aprovado
transformar_dim_tempo.py	Converte tipos, traduz mês/dia, boolean e ordena calendário	Aprovado
transformar_fato_venda.py	Resolve SKs, usa -1, converte medidas, calcula margem e protege o grão	Aprovado

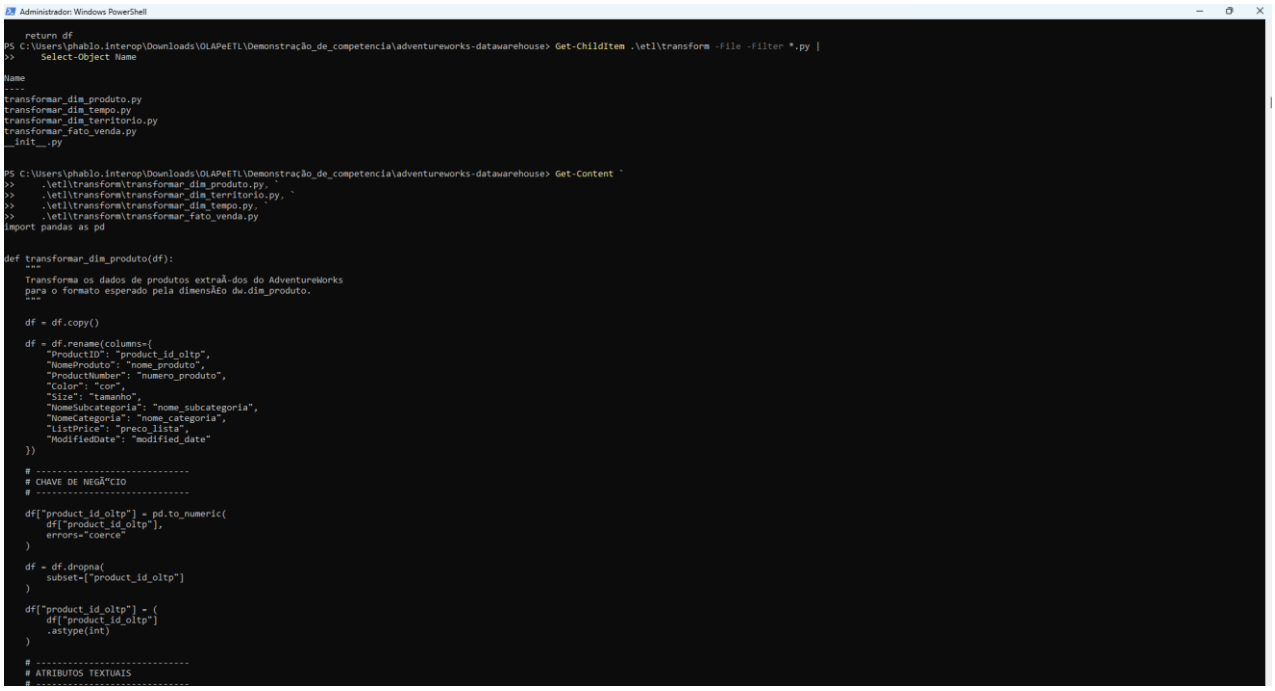
4.1 Correção identificada

Durante a revisão foi identificada uma incompatibilidade em transformar_dim_territorio.py. O campo codigo_pais possui tipo varchar(3) no DW, mas o tratamento de nulo utilizava o texto 'Não informado', que excederia o limite da coluna caso fosse realmente necessário aplicar o fallback.

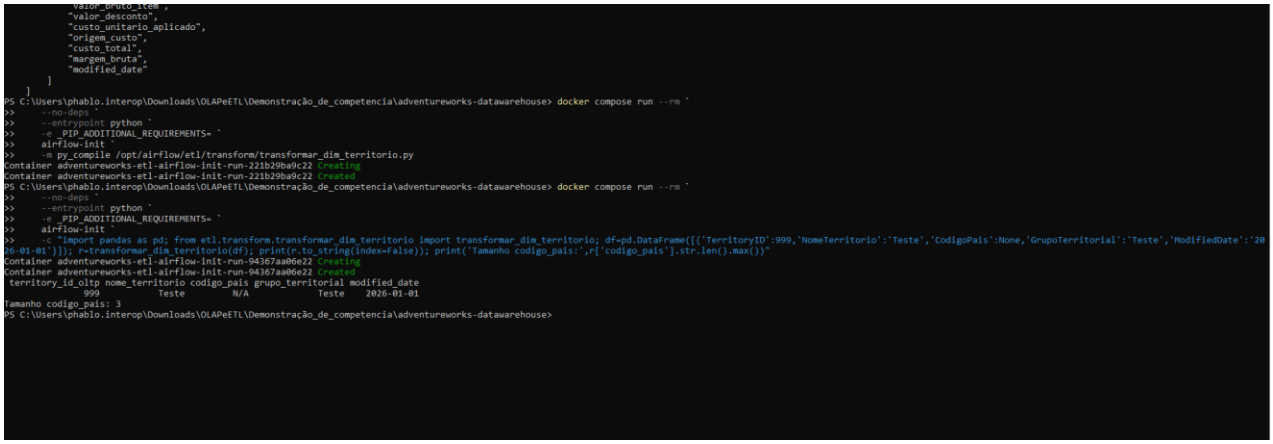
A regra foi alterada para:

```
df["codigo_pais"] = df["codigo_pais"].fillna("N/A")
```

A alteração foi validada com py_compile e com um DataFrame de teste contendo CodigoPais nulo. O resultado retornou N/A com comprimento 3, compatível com varchar(3).



Auditoria dos módulos Transform.



Validação da correção de codigo_pais para N/A (varchar(3)).

5. Revisão dos módulos Load

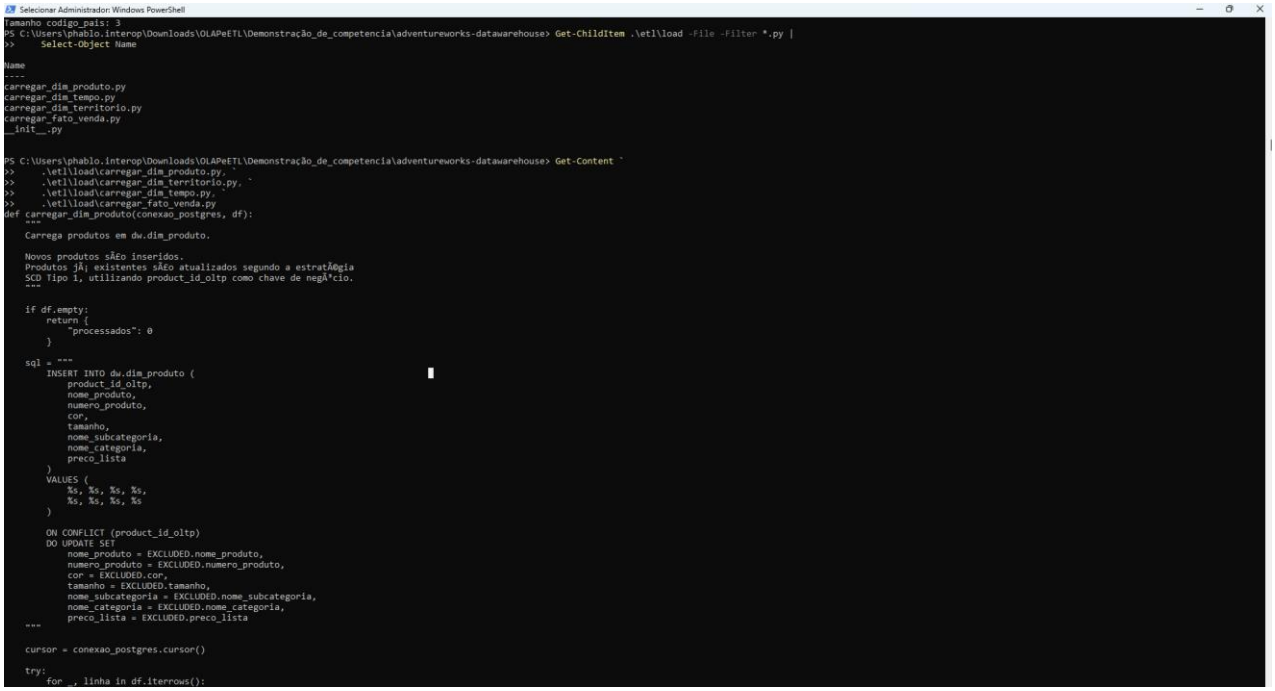
Os quatro loaders foram revisados para confirmar a correspondência com as tabelas do schema dw, o controle transacional e as estratégias de idempotência.

Módulo	Estratégia de carga	Finalidade	Status
carregar_dim_produto.py	ON CONFLICT (product_id_oltp) DO UPDATE	SCD Tipo 1	Aprovado
carregar_dim_territorio.py	ON CONFLICT (territory_id_oltp) DO UPDATE	SCD Tipo 1	Aprovado
carregar_dim_tempo.py	ON CONFLICT (sk_tempo) DO NOTHING	Carga determinística/idempotente	Aprovado
carregar_fato_venda.py	ON CONFLICT (sales_order_id, sales_order_detail_id) DO UPDATE	Preservar o grão e atualizar fatos	Aprovado

Todos os loaders utilizam commit após sucesso, rollback em caso de exceção e fechamento do cursor no bloco finally. Essa estrutura evita confirmação parcial de transações.

O loader da fato utiliza exatamente o par (sales_order_id, sales_order_detail_id) validado na Fase 4 como grão da tabela. Assim, novas execuções atualizam o item existente em vez de criar duplicidade.

Como melhoria futura, valores destinados a colunas NUMERIC poderiam ser enviados como Decimal em vez de float. Essa mudança não é necessária para o escopo atual, pois as cargas e totais foram validados com sucesso.



Auditoria dos módulos Load e estratégias ON CONFLICT.

6. Resultado consolidado

Bloco	Situação	Observação
Extract	Concluído	4 módulos compatíveis com o modelo final
Transform	Concluído	1 incompatibilidade corrigida e validada
Load	Concluído	4 loaders com ON CONFLICT e controle transacional
Referências ao modelo antigo	Nenhuma	Busca textual sem ocorrências
Validade sintática	Aprovada	compileall sem erros

7. Arquivo alterado durante a tarefa

A única alteração de código necessária durante esta revisão ocorreu em:

`etl/transform/transformar_dim_territorio.py`

Motivo: adequar o fallback de codigo_pais ao limite varchar(3), substituindo 'Não informado' por 'N/A'.

8. Conclusão

A tarefa “Revisar e adaptar módulos Extract/Transform/Load” foi concluída com sucesso. Os módulos ETL estão alinhados ao modelo dimensional validado na Fase 4, sem referências centrais ao modelo anterior. A estrutura Python compila no ambiente Docker, as extrações refletem as fontes corretas do AdventureWorks, as

transformações produzem os campos esperados pelo DW e os loaders implementam estratégias idempotentes compatíveis com SCD Tipo 1 e com o grão da fato.

A revisão identificou apenas uma incompatibilidade de comprimento em codigo_pais, que foi corrigida e testada. Com isso, o bloco Extract/Transform/Load está pronto para as próximas tarefas da Fase 5, voltadas a conexões, resolução de chaves e controle incremental por watermark.